

Using Claude Code for *DTC Competitive Analysis*

Five Claude Code prompts that build a competitive-intelligence context layer on any ecommerce category — plus the operator questions, patterns, and reusable toolkit that ship as a side effect.

Customer reviews are the *goldmine*.

Every creative strategist and ecom marketer worth listening to says the same thing: customer reviews are the cheapest, most honest research you will ever do. The buyer's actual language. What they wanted. What they got. What they wish was different — at scale, by the thousand, already public.

This guide uses your own brand's reviews — and your competitors' — to generate the rich insights that move **merchandising, acquisition mix, messaging, and pricing**. Reading 10,000 reviews by hand is impossible. Running a scraper without judgment produces a heap of CSVs nobody opens. Claude Code is the leverage in between — *if* you tell it exactly what to look for.

The interesting part isn't the code; it's what to *say*. Specificity is how you guard against AI slop — Claude defaults to readable-but-bland output unless you constrain what it produces. Five sequenced prompts, a series of operator questions on top, and you have a competitive-intelligence layer no SimilarWeb dashboard or \$15K agency deck can match.

IN THIS GUIDE

- 02 The five prompts, with example outputs inline
- 03 Your output folder, recapped
- 04 Operator questions to ask once context is loaded
- 05 The patterns — portable across competitive sets
- 06 The toolkit — what you've built by the end
- 07 The skills — install once, trigger anywhere
- 08 Amplify — connect your stack via MCPs
- 09 Try it on your own competitive set

HOW TO READ THIS GUIDE

Drop in **your own brand and 2–4 competitors** as you read — every prompt works against any list of ecommerce domains (clothing, beauty, supplements, pet, home — anywhere sites use third-party review widgets). For the worked examples, we use three DTC cookware brands — **carawayhome.com**, **fromourplace.com**, **hexclad.com** — illustratively only. The shape of output you'd produce on your own set is identical.

Step 1 — Build a *robust* review scraper.

A vague competitive brief gets a vague competitive scrape. The fastest way to upgrade Claude's response is to *refuse the safe answer* — pre-empt the bland, defensible one before it lands.

ASKED CLAUDE

I want to scrape ecommerce sites for reviews — multiple websites, reliably. **I don't want a "this particular one I couldn't fetch" outcome.** Reviews can be tricky because they're paginated, in iframes, or rendered late. Tell me the most reliable architecture, not the easiest one.

↩ to send

Send ↩

WHAT HAPPENED

Claude proposed a four-tier cascade — sitemap discovery, Playwright network interception, direct API replay, DOM fallback — instead of recommending one tool. The "no excuses" framing forced Claude to design for the failure modes, not the happy path. Without that constraint, the most likely answer would have been "use a scraping API" — which is correct but not interesting, and not what the consultant needed.

TAKE THIS WITH YOU

State what you'd reject before Claude can default to the safe answer. The phrase "I don't want a generic X" is one of the highest-leverage edits to a prompt.

Step 2 — Ask for an *HTML* and a *CSV* per product.

Concrete deliverables are constraints. *".html and .csv"* tells Claude what the next-step user — you, the analyst — will actually do with the data.

ASKED CLAUDE

Let's build this app. Output reviews per product as both **.html** and **.csv**. The HTML should be readable on its own; the CSV should be analysis-ready (one row per review). The toolkit should be smart enough to scan a whole site for reviews and decide what to extract.

↩ to send

Send ↩

WHAT HAPPENED

Two specific deliverables (**.html** and **.csv**) anchored the build. Claude wasn't guessing whether the consultant wanted a notebook, a dashboard, a database, or a JSON dump — they were told. The "scan the whole site" framing also pre-empted a common Claude failure: building a one-URL toy and calling it done.

TAKE THIS WITH YOU

Concrete outputs are constraints. ".html and .csv" told Claude how this would be used downstream — and Claude designed the output schema accordingly.

One page per product. *Thirty* of these.

Each product gets its own readable HTML with full review history, rating distribution, and verbatim customer quotes — exactly what Step 2 specified, rendered for any human (or Claude) to skim.

caraway-cookware-set

<https://carawayhome.com/products/cookware-set>

Reviews	Average rating	Distribution
4,012	4.78 / 5	5★ 3,318 4★ 410 3★ 142 2★ 78 1★ 64

Emily R. verified buyer

★★★★★

Best kitchen purchase I've made all year

Ordered the cream set after seeing it on Instagram for months. The pans heat evenly, eggs really do slide right off, and the magnetic rack means I'm not stacking them on top of each other anymore. The cookware finally matches my kitchen. Only friction: don't put them in the dishwasher — hand wash and they look new.

2026-04-18

Marcus W. verified buyer

★★★★★

Replaced my Teflon and not looking back

Doctor told me to lose the Teflon. Researched ceramic for weeks and went with Caraway. Six months in: nothing's chipped, nothing's stained, and the non-stick still works just as well as day one. Worth the price tag.

2026-04-09

Linda H. verified buyer

★★★★☆

Beautiful but heavy

These are gorgeous and they cook beautifully. My only honest note: they're heavier than I expected, especially the dutch oven. If you have wrist or shoulder issues, take that into account. Otherwise, the design is genuinely thoughtful — the magnetic rack is a small detail that I use every day.

2026-03-30

Colin K. verified buyer

★★★★★

Best purchase I've made in years

After years of using cheap cookware, I finally invested in Caraway. The quality is outstanding. The pans are perfect for everything I cook. The magnetic rack is a game-changer. I love the design and the fact that it's dishwasher safe. Worth every penny.

2026-04-15

Why this format: CSV is for analysis, HTML is for reading. When you (or Claude) need to understand *why* a product is outperforming or underperforming, the rendered review history — stars, dates, verified-buyer flags — is faster to skim than a spreadsheet. Both formats ship by default.

Step 3 — Make sure it works across *different review providers*.

When Claude says "this works on any review widget," your job is to find the competitor whose widget would prove it wrong. Run the off-happy-path test before you trust the data.

ASKED CLAUDE

Now test it on **fromourplace.com**.

↩ to send

Send ↩

...and a few minutes later:

ASKED CLAUDE

hexclad.com uses a different review platform. Try it.

↩ to send

Send ↩

WHAT HAPPENED

The first follow-up validated the cross-site claim within the same provider family. The second specifically targeted a different provider — and surfaced five real bugs in the scraper, including a URL-resolution edge case that returned 100 reviews instead of 14,000+ for the largest products. Without that one specific test, the toolkit would have shipped broken.

TAKE THIS WITH YOU

If Claude says "works generically," ask: "name a case where it might not." Then test that case.

Step 4 — Map *positioning* and the *personas* served.

With the dashboard in context, ask Claude what's driving each brand's positioning — and which personas each one best serves.

ASKED CLAUDE

Create an HTML that analyzes the **top 10 products by competitor**. Show review counts, average ratings, distribution, and category coverage. Side by side.

↩ to send

Send ↵

...then, with that dashboard in hand:

ASKED CLAUDE

Now I want to understand the **competitive dynamics** across the three brands. Create a report explaining each brand's positioning — **based on review volume, the makeup of top-10 products, and the language customers use**. Find which **personas each brand best serves** and why.

↩ to send

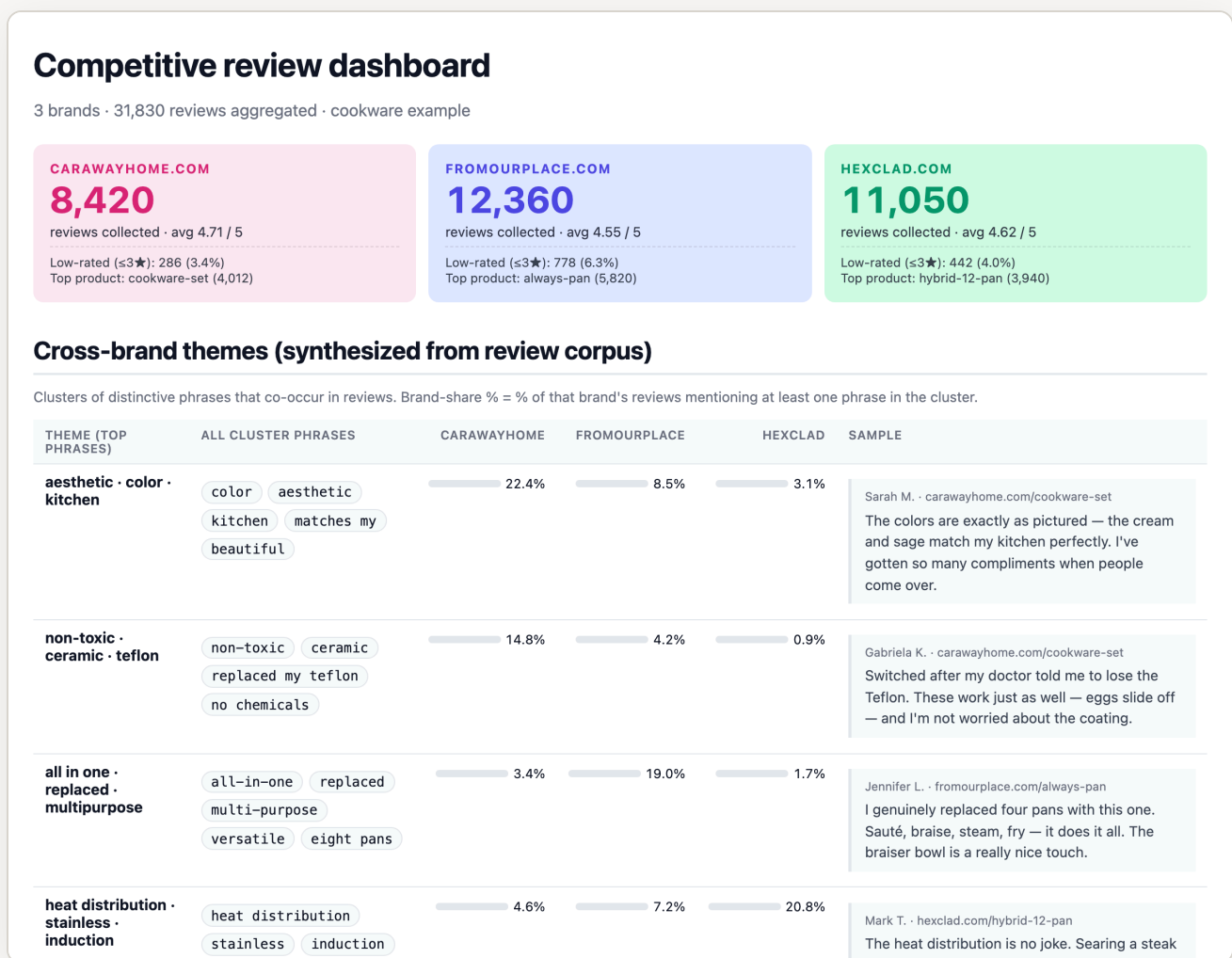
Send ↵

WHAT HAPPENED

Prompt 2 was only useful *because* Prompt 1 had already produced the dashboard. With the evidence in context, Claude's analysis cited specific quotes, distinctive phrases, and rating distributions — instead of speculating about what was happening. *Run the mechanical prompts first, even when the strategy is what you actually want.*

Here's what your *brand comparison* looks like.

A single page that compares review volume, rating quality, and corpus-synthesized themes across all three brands — with sample quotes inline. This is what Step 4's first prompt produces.



What's on screen: three stat cards (one per brand) with review volume, rating, and complaint rate; a cross-brand themes table with horizontal share-of-voice bars and a representative quote per theme. Themes are *not* hardcoded — they emerged from the actual review corpus by clustering distinctive phrases that co-occur in the same reviews.

You didn't just build pretty HTMLs — you built a *context layer*.

After the five prompts above, your working directory contains an entire competitive-intelligence dataset: thirty product pages, three brand summaries, a dashboard, and a report — readable on their own, and ready to load back into Claude as context for anything you'd want to ask next.

▶ output/	
▶ brand-a.com/	10 products · 4,021 reviews
● a-comforter.html	849 reviews · 4.81★
● a-comforter.csv	849 rows
● a-throw-1.html	330 reviews · 4.97★
● a-throw-1.csv	330 rows
... × 8 more product pairs	
● index.html	brand summary
▶ brand-b.com/	10 products · 38,145 reviews
... same structure (10 product HTML/CSV pairs + index)	
▶ brand-c.com/	10 products · 42,808 reviews
... same structure	
● _aggregate.json	distinctive phrases + cross-brand themes
● dashboard.html	cross-competitor comparison view
● report.html	narrative report with samples

What you're looking at: 30 product-detail HTMLs, 30 review CSVs, 3 brand indexes, a cross-brand dashboard, and a narrative report — all generated by one pipeline run. Every quote, every rating, every distinctive phrase is on disk and addressable.

WHY THIS MATTERS: THE RICH CONTEXT LAYER

This folder *is* the context layer. Open Claude Code in this directory and Claude can read every file. Ask any question — about a specific product, a specific phrase, a specific complaint — and Claude has the evidence pre-indexed. The question stops being "can I find this?" and becomes "what should I do with it?"

Questions an *expert ecommerce operator* would ask next.

With the folder loaded as Claude's context, the leverage is in what you ask. Most marketers stop at the dashboard. The operators get value out of the next ten prompts.

-
- 01** "How does each brand's **top-5 product mix** differ — hero-SKU-dependent vs balanced, premium vs entry, single-category vs cross-category? That's their strategic bet, made visible by what their customers actually buy."
-
- 02** "Which **customer persona is captured better by the competition** than by me? The persona competitors over-serve is usually a category gap they figured out before I did — and the persona *no* brand serves well is whitespace."
-
- 03** "Does any competitor sustain **high review volume without quality erosion**? The brand with both high volume *and* low complaint rate is the operationally hardest one to compete with — figure out why before scaling head-on."
-
- 04** "How concentrated is each competitor's review volume in their **#1 SKU**? A brand whose hero pulls 25%+ of top-10 review volume is hero-dependent — both a risk for them and a moat to be aware of for me."
-
- 05** "What **complaint pattern is unique** to each competitor's low-rated reviews? Where they overlap with my own product weaknesses → fix me. Where they're absent from my reviews → that's a gap I can exploit publicly."
-
- 06** "Which **language themes are present in one competitor's reviews and wholly absent in another's**? Absent themes are positioning whitespace — no brand is clearly serving that customer need yet."
-

OPTIONAL • DECK COMPOSITION

Optional — **compose the corpus into a *deck* and ship it.**

If your work ends in HTML, you can stop after the operator questions. If it ends in front of a client or a CMO, this last move composes the same evidence into a presentation. A finished competitive analysis is rarely "the answer" — it's HTML for the analyst, a deck for the room, a doc for the team. Reformat, don't redo.

ASKED CLAUDE

Create a **presentation of the doc** and **add it to my Google Drive**.

↩ to send

Send ↩

WHAT HAPPENED

Claude generated an 11-slide PowerPoint with consistent typography, embedded review quotes, and rating charts — then uploaded it via Google Drive Desktop sync. The deck wasn't reformatted from scratch; it was the same evidence base presented for a different audience. Composing outputs from a single corpus is one of Claude Code's strongest patterns.

TAKE THIS WITH YOU

Once the analysis exists in HTML, ask for the deck. Once the deck exists, ask for it in Drive. Each step is short because the substance is already there.

Eight patterns that sharpen any competitive analysis.

Stripped from the prompts above. Use them on any competitor set, in any vertical, with any review provider.

01 State what you'd reject.

"I don't want a generic answer that says X" is the fastest upgrade to any prompt. Claude defaults to the safe response unless told it isn't acceptable.

02 Anchor with concrete output formats.

".html and .csv," "an 11-slide deck," "a markdown doc with H2 sections" — concrete outputs constrain the work in ways that abstract goals can't.

03 Falsify the claim.

When Claude says "this works generically," your job is to find the case where it doesn't. Run the off-happy-path test before you trust the work.

04 Layer questions.

Mechanical prompt → analysis prompt → strategy prompt. Don't ask for "the strategic recommendation" before the data is in Claude's context.

05 Compose outputs across formats.

The same evidence base can become an HTML dashboard, a doc, a deck, an email, a slack message. Don't redo the work; reformat it.

06 Course-correct on framing, not just facts.

Claude's default is balanced — sometimes that flatters a comparison the data doesn't actually support. Tell Claude when the framing's wrong; that's permission to take a position.

07 Gate action with explanation.

"Explain how you'll do that before you action it." The single highest-leverage edit when the next step is non-trivial. Saves you hours of unwound work.

08 Defend privacy in shareable artifacts.

Claude doesn't infer NDAs. State explicitly what to redact, anonymize, or substitute when something is going public — before it gets published.

THE META-PATTERN

*Treat Claude as a **direct report**, not a **search engine**. Brief it like a smart colleague: what you want, what you've ruled out, what would be unacceptable, and what to check back on before doing anything irreversible.*

What you've actually built.

Five prompts, one folder, a reusable Python package that runs on any ecommerce category — plus a polished competitive analysis specific to your set.

The toolkit scrapes reviews from any ecommerce site by intercepting the network calls the site's own review widget makes — so it works across Yotpo, Judge.me, Okendo, Stamped, Loox, Bazaarvoice, and providers that haven't been invented yet. It then synthesizes distinctive language patterns across competitors directly from the corpus, with no hardcoded category vocabulary.

THE REPOSITORY, ROUGHLY

```
ecom-competitive-analysis/
├── README.md
├── CLAUDE.md           → operator instructions for Claude
├── prompts/           → reusable analysis playbooks
├── eca/               → the Python package
│   ├── crawler.py     → sitemap-based product URL discovery
│   ├── discovery.py   → Playwright network interception
│   ├── extractor.py    → direct API extraction with pagination
│   ├── aggregate.py   → distinctive-phrase synthesis
│   ├── dashboard.py   → cross-competitor HTML dashboard
│   ├── report.py      → narrative HTML report
│   └── platforms/     → per-platform product-id resolvers
└── docs/             → architecture, caveats, extensions
```

Run `eca pipeline brand-a.com brand-b.com brand-c.com` and the toolkit produces per-product CSVs, a cross-competitor dashboard, and a templated narrative report. The strategic interpretation is yours to write — the toolkit gives you the evidence.

A few hours of running prompts replaces what most agencies bill at **\$15–25K** for a single competitive deep-dive — and the next category you analyze takes minutes, not weeks, because the toolkit is reusable across verticals.

Or just install the *skills*.

Two Claude skills ship with the repo. Drop them into `~/.claude/skills/` and they trigger automatically when you talk to Claude about scraping reviews or running competitive analysis — no folder, no CLI flags, no setup beyond a one-time copy.

wonderland-review-scraper

triggers on *"scrape reviews from [site]"* · pulls CSV + HTML per product

wonderland-dtc-competitive-analysis

triggers on *"compare [brand A] vs [brand B]"* · runs the full pipeline + surfaces operator questions

INSTALL ONCE

```
git clone https://github.com/michaelwonderland/dtccompetitiveanalysis.git
cp -r dtccompetitiveanalysis/skills/wonderland-* ~/.claude/skills/
```

THEN JUST TALK TO CLAUDE

ASKED CLAUDE

Scrape reviews from carawayhome.com — top 10 products.

→ triggers **wonderland-review-scraper**

ASKED CLAUDE

Run a competitive analysis on carawayhome.com, fromourplace.com, hexclad.com.

→ triggers **wonderland-dtc-competitive-analysis**

Both skills *stop and ask* on judgment calls — anti-bot blocks, ToS thresholds, which strategic question to drill into. Same direct-report pattern: **you direct, Claude executes**.

You don't have to *stop* here.

The competitive review corpus is one layer. Connect your own operating reality through Claude's MCP integrations and the questions sharpen by an order of magnitude.

Inside Claude Code with the review corpus already loaded, you can plug in more of what your brand actually *does*. Each integration adds context Claude can cross-reference against the competitor reviews:

Drive your strategy decks, briefs, brand books, customer profiles

Notion OKRs, voice guidelines, customer-research notes

Shopify orders, customers, SKU-level revenue, cohort data

Motion your ad creative library and per-ad performance

Klaviyo segments, flows, engagement, attributable revenue

NOW THE QUESTIONS GET INTERESTING

- 01** "What's the difference between the buyer **Competitor A** attracts (per their reviews) and the buyer **we** attract (per our Shopify customers + Klaviyo segments)? Where's the underserved sub-segment we should be targeting?"

- 02** "Which of our SKUs solves the same job as **Competitor B's top three**? Cross-reference Shopify SKU revenue with their distinctive review themes — should we be merchandising any of ours harder on the PLP?"

- 03** "What ad creative is **Competitor C** running (Motion library) that maps to the praise themes in their reviews? Pull our brand voice from Notion and brief 3 of our own to test against."

- 04** "What landing-page styles does **Competitor A** have that we don't? Pull our top 5 SKUs from Shopify and propose new LPs that close the gap, using language patterns from their 5★ reviews."

- 05** "Which of our Klaviyo segments most resemble **Competitor C's** repeat-buyer reviewer cluster? Pitch a flow targeting them with messaging borrowed from Competitor C's strongest distinctive themes."

Each unanswerable with reviews alone, or with first-party data alone. The leverage is in the overlay.

Run the same play on your own competitive set.

The toolkit that came out of this conversation is open source. Clone it, point it at your category, and direct Claude through the same patterns.

The repository contains everything: the scraper, the corpus-synthesis engine, the cross-competitor dashboard, the templated narrative report, and a `CLAUDE.md` file that teaches Claude how to operate the toolkit on a fresh dataset. The repo is open source under MIT — clone it, fork it, modify it.

THE MINIMUM VIABLE WORKFLOW

1. Pick three competitors in your category.
2. Open the toolkit in Claude Code.
3. Ask Claude to *"run the competitive analysis on these three domains."*
4. When the dashboard and report are produced, ask the operator questions from the previous page.

YOU DON'T NEED THE REPO AT ALL.

The pipeline you've just seen is reproducible by following the prompts in this guide. Open Claude Code in any empty folder and walk through prompts 01–05 with your own competitive set — the toolkit will materialize as you go. The repo is a shortcut, not a requirement; the prompts are the actual product.

The substance of competitive intelligence isn't in the scraping or the dashboards — both have been commoditized. It's in the framing, the question you choose to ask, and the moments where you tell Claude its first answer wasn't the right one. Most marketers won't do this. The ones who will get an unfair compounding advantage.

Treat Claude as a *direct report*, not a *search engine*.

Brief it like a smart colleague: what you want, what you've ruled out, what would be unacceptable, and what to check back on before doing anything irreversible. That's the entire pattern. It compounds.

Most marketers prompt Claude like a chatbot — type a question, accept whatever comes back. The ones who direct it like a senior analyst — who push back, course-correct, and won't ship the flat first draft — compound an unfair advantage every quarter.

And resist the temptation to *streamline*. The point of this automation isn't to remove your thinking from the loop. It's to put more data *in front* of your thinking. Claude can produce the corpus. The strategic call is still yours to make.

You now have the playbook. Go pick three competitors.